DiagVoice TOGG Analiz API'si

Sürüm 1.2 · 11 Kasım 2026

1. Genel bakış

TOGG Analiz API'si, sürücüden alınan kısa bir ses kaydını analiz eder ve tek yanıtta iki sonuç döner:

- **Ses emosyonu**: dört duygu sınıfı (üzüntü, korku, mutluluk, öfke) için olasılıklar ve duygu yoğunluğu.
- **WURSS sağlık sinyali**: sesin sağlıklı / hasta sınıflarından hangisine yakın olduğu.

İki analiz sunucuda paralel çalışır. Konuşma metne çevrilmez; yalnızca ses sinyali kullanılır.

Özellik	Değer
Temel adres	`https://aiapi.diagvoice.com`
Uç nokta	`POST /v1/analyze/togg`
İstek biçimi	`multipart/form-data`
Yanıt biçimi	`application/json` (UTF-8)
Kimlik doğrulama	API anahtarı (bkz. Bölüm 2)
Tipik yanıt süresi	~0,25 sn (tek istek, 5 sn'lik kayıt; bkz. Bölüm 7)

2. Kimlik doğrulama

Her istekte DiagVoice tarafından verilen API anahtarı gönderilmelidir. İki başlıktan biri kullanılabilir:

```
Authorization: Bearer dv_sk_xxxxxxxx_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
X-API-Key: dv_sk_xxxxxxxx_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

- Anahtar `dv_sk_` ile başlar ve **gizlidir**. Yalnızca sunucu tarafında (TOGG backend'i) saklanmalıdır.
- **Anahtarı araç içi uygulamaya veya mobil uygulamaya gömmeyin**; uygulama paketinden çıkarılabilir. Önerilen akış: araç → TOGG backend'i → DiagVoice API.
- Anahtar sızdığından şüphelenirseniz DiagVoice ekibine bildirin; anahtar iptal edilip yenisi verilir.
- Anahtarın bilgileri ve limitleri `GET /v1/me` ile görülebilir.
- İsteklerde uygulamanızı tanıtan bir `User-Agent` başlığı gönderin (ör. `TOGG-Backend/1.0`). Bazı HTTP kütüphanelerinin varsayılan `User-Agent` değeri (ör. Python `urllib`) ağ katmanındaki bot korumasına takılıp `403` (gövdede `error code: 1010`) alabilir.

3. İstek

```
POST /v1/analyze/togg HTTP/1.1
Host: aiapi.diagvoice.com
Authorization: Bearer dv_sk_xxxxxxxx_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Content-Type: multipart/form-data; boundary=----X

------X
Content-Disposition: form-data; name="file"; filename="kayit.m4a"
Content-Type: audio/mp4

<ikili ses verisi>
------X--
```

Form alanları

Alan	Tür	Zorunlu	Açıklama
`file`	dosya	Evet	Ses kaydı. Desteklenen biçimler: WAV, M4A/AAC, MP3, FLAC, OGG, WEBM.

Ses kaydı önerileri

Konu	Öneri
Süre	3–30 saniye konuşma. Çok kısa kayıtlarda (< 2 sn) sonuçların güvenilirliği düşer.
Kanal / örnekleme	Mono, 16 kHz yeterlidir. Farklı biçimler sunucuda 16 kHz mono'ya dönüştürülür.
Biçim	16-bit PCM WAV veya AAC (M4A).
Boyut	En fazla 200 MB; pratikte 30 sn'lik bir kayıt 0,3–1 MB'tır.
Ortam	Mümkünse kabin gürültüsü düşükken ve tek konuşmacıyla.

Sunucunun kaydı nasıl işlediği

- **Ses emosyonu**: Sessiz bölümler ayıklanır (en fazla 30 sn konuşma tutulur), konuşma 10 sn'lik parçalara bölünür ve parça sonuçlarının ortalaması alınır. 10 sn'den kısa kayıtlar tek parça olarak işlenir.
- **WURSS**: Kaydın ortadaki en fazla 30 saniyesi kullanılır.

4. Yanıt

Başarılı bir istekte HTTP `200` döner.

```
{
  "audio": {
    "pred_id": 3,
    "pred_label": "anger",
    "probs": [0.2170, 0.2027, 0.1424, 0.4379],
    "intensity": 0.7416,
    "labels": { "0": "sadness", "1": "fear", "2": "happiness", "3": "anger" },
    "sample_rate": 16000,
    "duration_sec": 5.184,
    "num_segments": 1
  },
  "audio_error": null,
  "disease": {
    "pred_id": 1,
    "pred_label": "sick",
    "probs": [0.0168, 0.9832],
    "labels": { "0": "healthy", "1": "sick" }
  },
  "disease_error": null,
  "partial": false
}
```


Üst düzey alanlar

Alan	Tür	Açıklama
`audio`	nesne veya `null`	Ses emosyonu sonucu. Analiz başarısızsa `null`.
`audio_error`	metin veya `null`	Ses emosyonu analizi başarısızsa hata açıklaması; başarılıysa `null`.
`disease`	nesne veya `null`	WURSS sonucu. Analiz başarısızsa `null`.
`disease_error`	metin veya `null`	WURSS analizi başarısızsa hata açıklaması; başarılıysa `null`.
`partial`	mantıksal	`true`: iki analizden en az biri başarısız (bkz. Kısmi sonuç).

`audio` — ses emosyonu

Alan	Tür	Açıklama
`pred_label`	metin	En olası duygu: `sadness`, `fear`, `happiness`, `anger`.
`pred_id`	tam sayı	`pred_label`'ın sınıf numarası (0–3).
`probs`	sayı dizisi (4)	Sınıf olasılıkları; sıra `labels` ile aynıdır (0: sadness, 1: fear, 2: happiness, 3: anger). Toplamı 1'dir.
`intensity`	sayı (0–1)	Duygu yoğunluğu. 0 sakin, 1 çok yoğun.
`labels`	nesne	Sınıf numarası → etiket eşlemesi.
`sample_rate`	tam sayı	İşlenen örnekleme hızı (Hz). Her zaman 16000.
`duration_sec`	sayı	Sessizlik ayıklandıktan sonra analiz edilen ses süresi (sn).
`num_segments`	tam sayı	Analizde kullanılan 10 sn'lik parça sayısı (1–5).

`disease` — WURSS sağlık sinyali

Alan	Tür	Açıklama
`pred_label`	metin	`healthy` (sağlıklı) veya `sick` (hasta).
`pred_id`	tam sayı	0: healthy, 1: sick.
`probs`	sayı dizisi (2)	Sırasıyla healthy ve sick olasılıkları. Toplamı 1'dir.
`labels`	nesne	Sınıf numarası → etiket eşlemesi.

> WURSS sonucu, ses özelliklerine dayalı bir **gösterge**dir; tıbbi teşhis değildir ve tek başına sağlık kararı için kullanılmamalıdır.

Kısmi sonuç

İki analiz birbirinden bağımsızdır. Biri başarısız olsa bile yanıt **HTTP 200** ile döner; `partial` alanı `true` olur, başarısız olan bölüm `null`, ilgili `*_error` alanı doludur:

```
{
  "audio": { "pred_label": "happiness", "...": "..." },
  "audio_error": null,
  "disease": null,
  "disease_error": "<hata açıklaması>",
  "partial": true
}
```

`partial == false` ise iki sonuç da geçerlidir. `partial == true` ise her bölümü ayrı kontrol edin: `audio_error == null` ise emosyon sonucu, `disease_error == null` ise WURSS sonucu geçerlidir. `*_error` metninin içeriği değişebilir; programatik karar için yalnızca `null` olup olmadığına bakın.

5. Hata durumları

Hatalarda gövde her zaman aynı biçimdedir:

```
{
  "error": {
    "code": "rate_limited",
    "message": "Dakikalık istek sınırı aşıldı.",
    "request_id": "3f9c2a7d1b4e8f60"
  }
}
```

Programatik kararlar için `code` alanını kullanın; `message` metni değişebilir. Destek taleplerinde `request_id` değerini iletin (aynı değer `X-Request-ID` yanıt başlığında da bulunur).

HTTP	`code`	Ne zaman	Önerilen davranış
`200`	—	İstek işlendi (tam veya kısmi sonuç)	`partial` alanını kontrol edin.
`401`	`missing_api_key`	Anahtar gönderilmedi	Başlığı ekleyin.
`401`	`invalid_api_key`	Anahtar hatalı	Anahtarı kontrol edin; tekrar denemeyin.
`401`	`key_revoked`, `key_expired`	Anahtar iptal edilmiş veya süresi dolmuş	DiagVoice ekibinden yeni anahtar isteyin.
`403`	`scope_denied`	Anahtarın bu uca yetkisi yok	DiagVoice ekibiyle iletişime geçin.

HTTP	code	Ne zaman	Önerilen davranış
403	client_disabled	Hesap erişimi kapalı	DiagVoice ekibiyle iletişime geçin.
422	invalid_request	file alanı yok veya istek biçimi hatalı	İsteği düzeltin; tekrar denemeyin.
413	—	Dosya 200 MB'tan büyük	Kaydı kısaltın.
429	rate_limited	Dakikalık istek sınırı aşıldı	Retry-After saniye bekleyip tekrar deneyin.
429	concurrency_limited	Aynı anda izin verilenden fazla istek	Devam eden bir istek bitince tekrar deneyin.
429	quota_exceeded	Aylık kota doldu	DiagVoice ekibiyle iletişime geçin.
500	internal_error	Sunucuda beklenmeyen hata	Bir kez tekrar deneyin.
502, 503, 504	upstream_error, service_unavailable, upstream_timeout	Sunucu geçici olarak erişilemiyor veya yoğun	Artan bekleme ile tekrar deneyin.
524	—	Yanıt 100 sn içinde üretilemedi (aşırı yoğunluk)	Artan bekleme ile tekrar deneyin veya iş API'sini kullanın (Bölüm 9).
404	not_found	İş bulunamadı (süresi dolmuş veya başka anahtara ait)	job_id'yi kontrol edin.
429	too_many_jobs	Anahtarın bekleyen iş sınırı doldu	Bekleyen işler bitince tekrar deneyin.
503	queue_full	İş kuyruğu dolu	Retry-After saniye bekleyip tekrar deneyin.

Yanıt başlıkları

Başlık	Anlamı
X-Request-ID	İsteğin takip numarası (her yanıtta)
X-RateLimit-Limit	Dakikalık istek sınırı
X-RateLimit-Remaining	Bu dakika içinde kalan istek hakkı (yaklaşık)
Retry-After	429 yanıtlarında: tekrar denemeden önce beklenecek süre (sn)

Zaman aşımı ve tekrar deneme

- İstemci zaman aşımını **30 saniye** olarak ayarlamanızı öneririz. Normal koşullarda yanıt 1 saniyenin altındadır.
- Tekrar deneme yalnızca 429 (Retry-After kadar bekleyerek), 500, 502, 503, 504, 524 ve ağ hatalarında yapılmalıdır. En fazla 3 deneme, aralarında artan bekleme önerilir (ör. 1 sn, 2 sn, 4 sn).
- Kısmi sonuçta (partial == true) aynı kayıt yeniden gönderilebilir; bu da en fazla bir kez yapılmalıdır.

6. Örnekler

cURL

```
curl -X POST https://aiapi.diagvoice.com/v1/analyze/togg \
  -H "Authorization: Bearer $DIAGVOICE_API_KEY" \
  -F "file=@kayit.m4a;type=audio/mp4"
```

Python

```
import os
import requests

API_KEY = os.environ["DIAGVOICE_API_KEY"]

with open("kayit.m4a", "rb") as f:
    r = requests.post(
        "https://aiapi.diagvoice.com/v1/analyze/togg",
        headers={"Authorization": f"Bearer {API_KEY}"},
        files={"file": ("kayit.m4a", f, "audio/mp4")},
        timeout=30,
    )
if r.status_code != 200:
    err = r.json().get("error", {})
    raise RuntimeError(f"{r.status_code} {err.get('code')}: {err.get('message')} (request_id={err.get('request_id')})

body = r.json()
if body["audio_error"] is None:
    print("Duygu:", body["audio"]["pred_label"], "yoğunluk:", body["audio"]["intensity"])
if body["disease_error"] is None:
    print("WURSS:", body["disease"]["pred_label"])
```

Kotlin (OkHttp) — TOGG backend'i

```
val client = OkHttpClient.Builder()
    .callTimeout(30, TimeUnit.SECONDS)
    .build()

val body = MultipartBody.Builder()
    .setType(MultipartBody.FORM)
    .addFormDataPart(
        "file", "kayit.m4a",
        File(path).asRequestBody("audio/mp4".toMediaType())
    )
    .build()

val request = Request.Builder()
    .url("https://aiapi.diagvoice.com/v1/analyze/togg")
    .header("Authorization", "Bearer $apiKey")   // anahtar sunucu tarafında saklanır
    .post(body)
    .build()

client.newCall(request).execute().use { resp ->
    val json = JSONObject(resp.body!!.string())
    if (!resp.isSuccessful) {
        val err = json.getJSONObject("error")
        error("HTTP ${resp.code} ${err.getString("code")} (request_id=${err.getString("request_id")})")
    }
    if (json.isNull("audio_error")) {
        val audio = json.getJSONObject("audio")
        println("Duygu: ${audio.getString("pred_label")}")
    }
    if (json.isNull("disease_error")) {
        val disease = json.getJSONObject("disease")
        println("WURSS: ${disease.getString("pred_label")}")
    }
}
```

7. Performans

10 Kasım 2026'da üretim sunucusunda ölçülmüştür (5 sn'lik, 150 KB M4A kayıt):

Senaryo	Medyan yanıt süresi	p95
Tek istemci, ardışık istekler	0,25 sn	0,27 sn
10 eşzamanlı istemci	1,6 sn	1,7 sn
50 eşzamanlı istemci	8,6 sn	9,0 sn

Süreler ağ gecikmesini ve ses yüklemesini içerir.

Kayıt uzunluğu ve boyutu süreyi doğrudan etkiler. Örneğin 49 sn'lik, 48 kHz WAV (4,7 MB) bir kayıtta sunucu tarafı işlem ~3 sn sürmüş, yükleme süresi buna eklenmiştir. Hızlı yanıt için:

- Kaydı 30 sn ile sınırlayın (sunucu zaten en fazla 30 sn kullanır).
- Göndermeden önce 16 kHz mono'ya indirin veya AAC (M4A) kullanın; bu, dosya boyutunu 5–10 kat küçültür.

8. Limitler ve kullanım kuralları

Limitler anahtar başınadır ve anahtar oluşturulurken DiagVoice tarafından belirlenir; güncel değerler `GET /v1/me` yanıtındadır.

Limit	Varsayılan
Dakikalık istek	120
Aynı anda işlenen istek	5
Aylık istek kotası	Sözleşmeye göre

- Aynı kaydı paralel olarak birden çok kez göndermeyin.
- Analiz edilecek her ses kaydı için bir istek gönderin; sürekli akış (streaming) desteklenmemektedir.
- Kayıtlar yalnızca analiz amacıyla iletilmelidir; kayıt içeriğinde kişisel bilgi bulunmamasına özen gösterin.

9. Asenkron iş API'si (isteğe bağlı)

Normal kullanımda `POST /v1/analyze/togg` yeterlidir (yanıt ~0,25 sn). Uzun kayıtlarda, yoğun saatlerde veya sonucu beklemek istemediğiniz durumlarda işi kuyruğa gönderip sonucu sonradan alabilirsiniz. Bağlantı kopsa bile iş kaybolmaz.

Uç	Açıklama
`POST /v1/jobs`	Form: `file`, `kind=togg`, isteğe bağlı `webhook_url`. Sorgu: `wait` (0–30 sn).
`GET /v1/jobs/{job_id}`	Durum: `queued`, `running`, `succeeded`, `failed`, `cancelled`. Sorgu: `wait` (0–30 sn).
`DELETE /v1/jobs/{job_id}`	Henüz başlamamış işi iptal eder.

```
curl -X POST "https://aiapi.diagvoice.com/v1/jobs?wait=10" \
  -H "Authorization: Bearer $DIAGVOICE_API_KEY" \
  -F "kind=togg" -F "file=@kayit.m4a;type=audio/mp4"
```

- İş `wait` süresi içinde biterse **200** ve sonuç döner; bitmezse **202**, `job_id` ve kuyruktaki sıra (`position`) döner.
- Sonuç `result` alanında, `POST /v1/analyze/togg` yanıtıyla aynı biçimdedir. Hata durumunda `error` alanı doludur.
- Sonuçlar 24 saat saklanır.
- **Webhook**: `webhook_url` verilirse iş bitince bu adrese JSON `{job_id, status, result, error}` POST edilir. Adres `https://` ile başlamalı ve herkese açık olmalıdır. İstek gövdesi `X-DiagVoice-Signature: sha256=<HMAC-SHA256>` başlığıyla imzalanır; imza anahtarı DiagVoice ekibince ayrıca iletilir.

10. Sürümleme ve geçiş

- `/v1` altındaki uçlarda mevcut alanların adı ve anlamı değişmez. Yanıtlara yeni alanlar eklenebilir; istemcilerin bilinmeyen alanları yok sayması gerekir.
- **Eski uç (`POST /togg-analyze`)**: Geçiş süresince anahtarsız çalışmaya devam eder; yanıtında `partial` alanı yoktur. Geçiş tamamlandığında bu uç kapatılacak, öncesinde bildirim yapılacaktır. Yeni entegrasyonlarda yalnızca `/v1/analyze/togg` kullanın.

11. İletişim

Teknik sorular, anahtar talepleri ve entegrasyon desteği için DiagVoice ekibiyle iletişime geçin.